DataBase Assignment

EE 470 - Introduction to the Internet of Things- Fall / Version: V2

NAME _Antonio Kassis_ DATE: _10/28/2026_

This project must be done individually!

Link to this document:
https://docs.google.com/document/d/1zhLaeiWjtqfpCTeJzd_n543yl6_SxahBxWd4lHh4srY/edit?usp=sharing

Please make sure you submit your assignment individually!

DataBase Assignment

EE 470 - Introduction to the Internet of Things- Fall / Version: V2

Assignment Overview

The purpose of this exercise is to become familiar with MySQL database and its commands.

1. Learning Objectives

- Set Up an online web page
- Learn about SQL
- Practice SQL commands
- Create a Table for sensor data
- Using Github.

2. Required Materials & Reference Information

- Signup for https://hostinger.com
- database tutorial available on the course GitHub
- Tutorial on creating an account with hostinger.com
- You can find the longitude and latitude format here
- Good tutorial: https://www.tutorialspoint.com/mariadb/mariadb_introduction.htm

3. Experiment

This experiment has TWO parts.

Part 1:
In this part of the experiment you need to signup for an account at https://hostinger.com
This will allow you to have your personal web page and database.

Part 2:
In the second part of the experiment you will need to access the database using a PHP file and visualize the data using the Chartjs library.

DataBase Assignment

EE 470 - Introduction to the Internet of Things- Fall / Version: V2

PART 1

1- A Short Introduction to Databases

A database is a systematic or organized collection of related information that is stored in such a way that it can be easily accessed, retrieved, managed, and updated. It is where all data is stored, very much like a library that houses a wide range of books from different genres. Think of data as books.

In a database, you can organize the data in rows and columns in the form of a table. Indexing the data makes it easy to find and retrieve it again as and when required. Many websites on the World Wide Web are managed with the help of databases. To create a database so that the data is accessible to users through only one set of software programs, database handlers are used.

MySQL, SQL Server, MongoDB, Oracle Database, PostgreSQL, Informix, Sybase, etc. are all examples of different databases. These modern databases are managed by DBMS. Structured Query Language, or SQL as it is more widely known, is used to operate on the data in a database.

MariaDB is a popular fork of MySQL created by MySQL's original developers. It grew out of concerns related to MySQL's acquisition by Oracle. It offers support for both small data processing tasks and enterprise needs. It aims to be a drop-in replacement for MySQL requiring only a simple uninstall of MySQL and an install of MariaDB. MariaDB offers the same features of MySQL and much more.

For more information about Databases read this: https://intellipaat.com/blog/what-is-database/

1.1. Create an Account

A- Carefully review the Tutorial on creating an account with hostinger.com. Create a very simple webpage, as described in the tutorial. Make sure your web page is working. Add your name to your webpage. We will call this web page Internet of Things (IoT). We will use this web page to place links to all your future assignments.

https://antoniokassis.com/firstpage.html
Link to your webpage

DataBase Assignment



Figure 1: Snapshot of my webpage

B- Create a dBase. Make sure your database's name is your `YourFirstNameLastName`. The username for the database must be `db_YourFirstNameLastName`. For example database: `FaridFarahmand`
User: `db_FaridFarahmand`

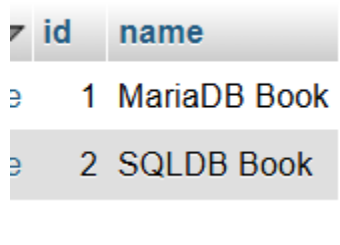
You should see something like this:

Database
information_schema
u301268454_AnthonyMeyer

Follow the Tutorial and create a simple `BOOK` table. Using `SHOW DATABASES` you can show your dBase. For more information Please refer to Github Tutorial. Your final `BOOK TABLE` should look something like this:

id	name
1	MariaDB Book
2	SQLDB Book

Take a snapshot of your results:



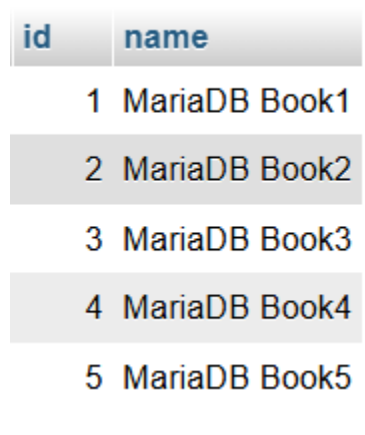
id	name
1	MariaDB Book
2	SQLDB Book

Figure 2: Snapshot of BOOK table.

1.2. Practice with MySQL Commands

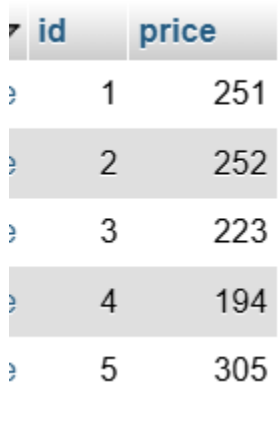
Using the phpMyAdmin in hostinger.com let's practice more with MySQL commands.

Review the commands in the database tutorial available on the course GitHub. Make sure you complete the tutorial. Once you complete the tutorial you should have a database table with your name and TWO tables having: `Price and Book`. Each table must have the appropriate elements.



id	name
1	MariaDB Book1
2	MariaDB Book2
3	MariaDB Book3
4	MariaDB Book4
5	MariaDB Book5

Figure 3a: Screenshot of Book table.



id	price
1	251
2	252
3	223
4	194
5	305

Figure 3b: Screenshot of Price table.

Now let's combine different fields and create ONE single table showing both book name and price - similar to below:

name	price
MariaDB Book1a	250
MariaDB Book2	252
MariaDB Book3	223
MariaDB Book4	194
MariaDB Book5	305

Take a snapshot of your results:

name	price
MariaDB Book1	251
MariaDB Book2	252
MariaDB Book3	223
MariaDB Book4	194
MariaDB Book5	305

Figure 4: Screenshot of joined Book and Price table

1.3. Create Your Own Sensory-Table

Now you need to design your own database using the following **design requirements**:

Design Two tables: `sensor_data & sensor_resister.`

`sensor_data` must have the following fields:

- node_name <up to 10 characters>; this will be used as the KEY
- time_received <indicating time/date>
- temperature <a numeric value between -10 to +100 celsius with maximum of 6 positions>
- humidity <a number value between 0-100 with maximum of 6 positions>

`sensor_register` must have the following fields:

- node_name <up to 10 characters>; this will be used as the KEY
- manufacturer <up to 10 characters>
- longitude < <a real (positive/negative number with maximum of 15 positions>
- Latitude <a real (positive/negative number with maximum of 15 positions>

You can find the longitude and latitude format here.

You can use a command similar to the following to create sensor data:

```
INSERT INTO sensor (node_name, time_received, temperature, humidity)
VALUES ('node-1','10-05-2022 10:05:34',84,25);
```

Your database must have the following requirements:

1- The starting data must be October 1, 2022, at 11:00 am.
2- The data must be sent every half an hour.
3- We need to register 5 different sensors with five different manufacturer names.
5- Each sensor must transmit at least 4 different data sets. This means there must be at least 4 different entries from each sensor node.
6- Node name format must be the following: node_1, node_2, node_3…. Etc.
7- No data from the sensor_data table is accepted if the transmitting node is not registered in the sensor_register table.
8- Create a VIEW of both tables and call it ensor_combined.

Once you have completed the database construction and the VIEW, take a snapshot of the content of the VIEW (sensor_combined) and all the commands you used to construct the databases. In addition, take a snapshot of the database structure, as it appears on the PhpMyAdmin portal, as shown below. Your snapshots should be similar to the figures below.

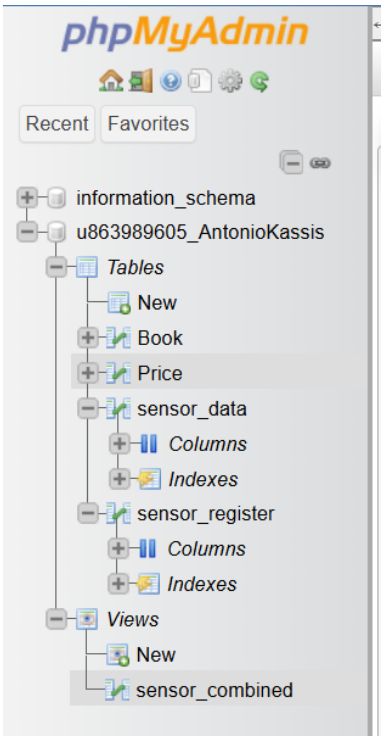


Figure 5a: Screenshot of the database structure

```sql
CREATE TABLE sensor_register (
    node_name VARCHAR(10) NOT NULL,
    manufacturer VARCHAR(10) NOT NULL,
    longitude DECIMAL(15,10) NOT NULL,
    latitude DECIMAL(15,10) NOT NULL,

    PRIMARY KEY (node_name)
);

CREATE TABLE sensor_data (
    id INT NOT NULL AUTO_INCREMENT,
    node_name VARCHAR(10) NOT NULL,
    time_received DATETIME NOT NULL,
    temperature DECIMAL(6,2) NOT NULL,
    humidity DECIMAL(6,2) NOT NULL,

    PRIMARY KEY (id),

    UNIQUE (node_name, time_received),

    CONSTRAINT fk_sensor_node
        FOREIGN KEY (node_name)
        REFERENCES sensor_register(node_name),

    CHECK (temperature >= -10 AND temperature <= 100),
    CHECK (humidity >= 0 AND humidity <= 100)
);
```

Figure 5b: Screenshot of the table commands

node_name	manufacturer	longitude	latitude	time_received	temperature	humidity
node_1	Bosch	-122.714100	38.339600	2022-10-01 11:00:00	22.500	45.000
node_1	Bosch	-122.714100	38.339600	2022-10-01 11:30:00	23.000	46.000
node_1	Bosch	-122.714100	38.339600	2022-10-01 12:00:00	23.500	47.000
node_1	Bosch	-122.714100	38.339600	2022-10-01 12:30:00	24.000	48.000
node_2	Siemens	-122.713500	38.340100	2022-10-01 11:00:00	20.000	50.000
node_2	Siemens	-122.713500	38.340100	2022-10-01 11:30:00	20.500	51.000
node_2	Siemens	-122.713500	38.340100	2022-10-01 12:00:00	21.000	52.000
node_2	Siemens	-122.713500	38.340100	2022-10-01 12:30:00	21.500	53.000
node_3	Honeywell	-122.712900	38.340600	2022-10-01 11:00:00	25.000	40.000
node_3	Honeywell	-122.712900	38.340600	2022-10-01 11:30:00	25.500	41.000
node_3	Honeywell	-122.712900	38.340600	2022-10-01 12:00:00	26.000	42.000
node_3	Honeywell	-122.712900	38.340600	2022-10-01 12:30:00	26.500	43.000
node_4	DHT	-122.712300	38.341100	2022-10-01 11:00:00	27.000	38.000
node_4	DHT	-122.712300	38.341100	2022-10-01 11:30:00	27.500	39.000
node_4	DHT	-122.712300	38.341100	2022-10-01 12:00:00	28.000	40.000
node_4	DHT	-122.712300	38.341100	2022-10-01 12:30:00	28.500	41.000
node_5	Sensirion	-122.711700	38.341600	2022-10-01 11:00:00	18.000	55.000
node_5	Sensirion	-122.711700	38.341600	2022-10-01 11:30:00	18.500	56.000
node_5	Sensirion	-122.711700	38.341600	2022-10-01 12:00:00	19.000	57.000
node_5	Sensirion	-122.711700	38.341600	2022-10-01 12:30:00	19.500	58.000

Figure 5c: Screenshot of the sensor_combined VIEW's values

Prove that if you insert data for a sensor that is NOT registered the value will not be accepted.

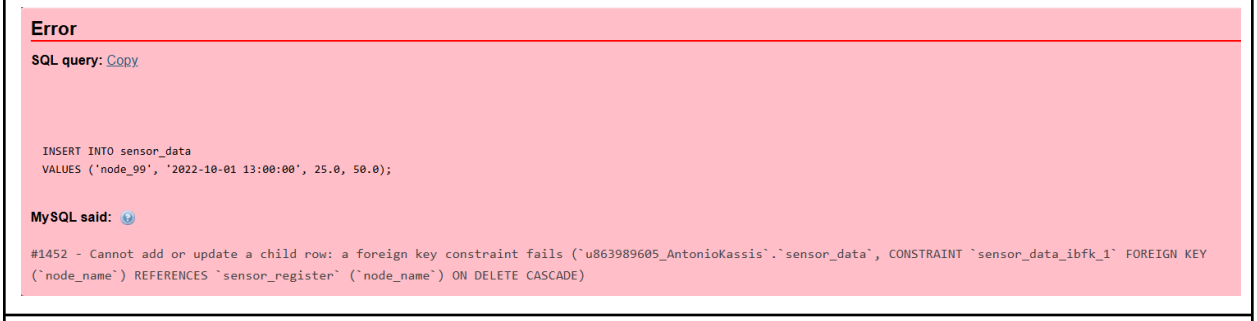


Figure 6: Screenshot of the constraints stopping an invalid tuple into the table

Please note: During your demonstration, you must show that all assigned features are fully functional. Each missing or non-functioning feature will result in a **7-point deduction** from your grade.

Then, in the provided space below, provide the link to that web page. This page will serve as your **main web page for the remainder of the semester**, and all assignment links must be posted there. This is a simple template I have created: https://faridfarahmand.net/SampleStudentAssignmentIndex.html

You can copy the template page and easily update it.

https://antoniokassis.com/assignments.html
Your web page link.

2- Answer the following questions

Answer the following questions.

1. What type of database does hostinger.com use?
   Hostinger uses MariaDB, which is a relational database management system that is compatible with MySQL.

2. What type of web server does hostinger.com use?
   Hostinger uses the LiteSpeed Web Server.

3. Referring to the data type tutorial (https://www.tutorialspoint.com/mariadb/mariadb_data_types.htm) assuming we have the following:
   ```
   CREATE TABLE ints (a INT,b INT UNSIGNED,c INT ZEROFILL);
   ```

   Would this insertion command work? `INSERT INTO ints VALUES (-10,-10,-10);`
   No. The value -10 can be stored in a, but b is UNSIGNED, so it cannot store negative values. ZEROFILL is also intended for non-negative values.

4. What is the purpose of having a primary key in a dBase?
   A primary key uniquely identifies each row in a table and prevents duplicate records from using the same key value.

5. What is the purpose of having a foreign key in a dBase?
   A foreign key creates a relationship between two tables and helps make sure referenced data exists in the related table.

6. Can a database have multiple tables?
   Yes. A database can contain many tables, and those tables can be related to each other.

7. What is a relational database?
   A relational database stores data in tables made of rows and columns. The tables can be linked together using keys.

8. Is MariadBase a relational or non-relational dBase?
   MariaDB is a relational database.

9. What is a database schema?
   A database schema is the structure or design of a database. It defines the tables, columns, data types, keys, and relationships.

10. What is a Table in a relational Database?
    A table is a collection of related data organized into rows and columns.

11. What are the main components of a Table in a relational database?
    The main components are rows, columns, data types, keys, and constraints.

12. In terms of data type, what is the difference between char(size) and varchar(size)?
    CHAR(size) stores fixed-length text, while VARCHAR(size) stores variable-length text up to the maximum size specified.

13. What would be an example of datetime datatype format?
    YYYY-MM-DD HH:MM:SS

3. GitHub

- Please post your code (PHP, SQL, etc.) in your GitHub. Make sure it is clear which assignment the code belongs to. All codes must have comments.
- Please add the link to your GitHub repository on the web page you created in **Section 1.1**.

Link to your GitHub: https://github.com/Neo4724/IoT_EE_470/tree/main/EE%20470%20-%20Assignment%204%3A%20Part%201%20-%20[PHP%20%26%20Database]

4. Quick Survey

Answer the following questions, please:

Survey question	Response
On a scale of 1-10 how did you like this exercise? (10 is the best, 1 is the worst)	8
On a scale of 1-10 how much did you learn as a result of completing this exercise? (10 = plenty; 1=very little)	7
How many hours did you spend completing this exercise?	9

END OF PART 1

PLEASE READ THE REQUIREMENTS CAREFULLY! If you are unable to answer questions during your demo it is considered an incomplete demo. Each incomplete demo will receive **-15 points**. Make sure you understand how your code works! Understand the structure of your design.

PART 2

Before attempting this part make sure you review the lecture slides. Also, make sure you have downloaded the Chart.js from the course GitHub site (https://github.com/faridfarahmand/SSU/tree/main/IoTCourse/Chartjs).

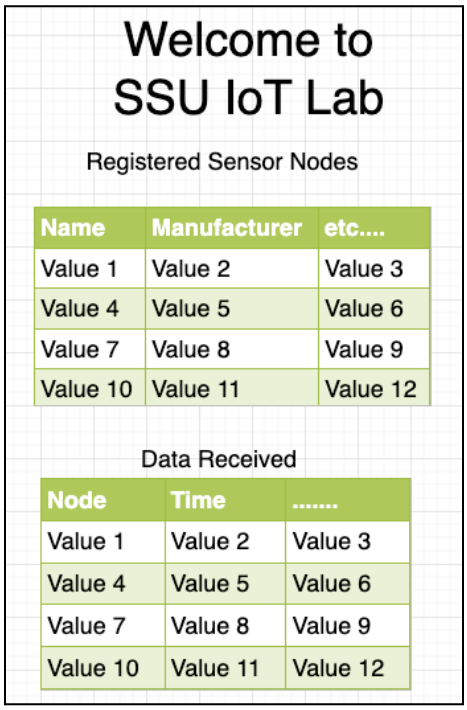
All files must be placed on hostinger.com. For faster interaction with the host, use FTP (e.g., Filezilla) to upload your files.

1- Connect and Display Using PHP

In this section we are using your sensor_data and sensor_resister tables that you created in PART 1.

Write a PHP program that accesses your sensor_data and sensor_resister tables and displays the values in two separate tables similar to the figure below. **Note that your colors, background, and page format can be different. However, all the values in the database must be clearly displayed.** When displaying the values, all displayed values must be sorted by Node Name and then Time.

DataBase Assignment



Welcome to SSU IoT Lab

Registered Sensor Nodes

Name	Manufacturer	etc....
Value 1	Value 2	Value 3
Value 4	Value 5	Value 6
Value 7	Value 8	Value 9
Value 10	Value 11	Value 12

Data Received

Node	Time	.....
Value 1	Value 2	Value 3
Value 4	Value 5	Value 6
Value 7	Value 8	Value 9
Value 10	Value 11	Value 12

```php
<?php

header('Content-Type: application/json');

// Your Hostinger database information
define('DB_HOST', 'localhost');
define('DB_USERNAME', 'u863989605_db_AntonioKass');
define('DB_PASSWORD', 'Blastwing472');
define('DB_NAME', 'u863989605_AntonioKassis');

// Connect to database
$mysqli = new mysqli(
    DB_HOST,
    DB_USERNAME,
    DB_PASSWORD,
    DB_NAME
);

// Check connection
if ($mysqli->connect_error) {
    die("Connection failed: " . $mysqli->connect_error);
}

// Get data from one sensor
$query = "
    SELECT node_name, temperature, humidity, time_received
    FROM sensor_data
    WHERE node_name = 'node_1'
    ORDER BY time_received
";

$result = $mysqli->query($query);

// Store results
$data = array();

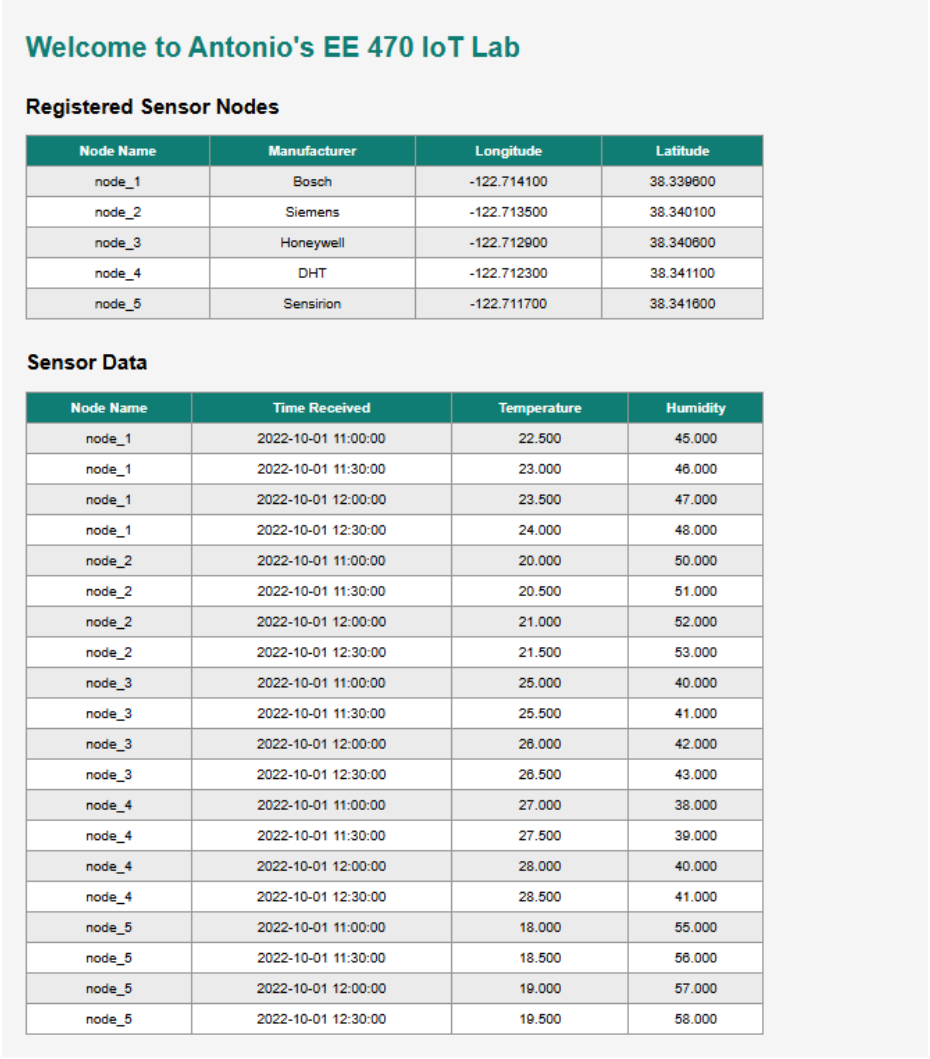
while ($row = $result->fetch_assoc()) {
    $data[] = $row;
}

// Close connection
$mysqli->close();

// Output JSON
echo json_encode($data);

?>
```

Figure 1: Screenshot of the php code



Welcome to Antonio's EE 470 IoT Lab

Registered Sensor Nodes

Node Name	Manufacturer	Longitude	Latitude
node_1	Bosch	-122.714100	38.339600
node_2	Siemens	-122.713500	38.340100
node_3	Honeywell	-122.712900	38.340600
node_4	DHT	-122.712300	38.341100
node_5	Sensirion	-122.711700	38.341600

Sensor Data

Node Name	Time Received	Temperature	Humidity
node_1	2022-10-01 11:00:00	22.500	45.000
node_1	2022-10-01 11:30:00	23.000	46.000
node_1	2022-10-01 12:00:00	23.500	47.000
node_1	2022-10-01 12:30:00	24.000	48.000
node_2	2022-10-01 11:00:00	20.000	50.000
node_2	2022-10-01 11:30:00	20.500	51.000
node_2	2022-10-01 12:00:00	21.000	52.000
node_2	2022-10-01 12:30:00	21.500	53.000
node_3	2022-10-01 11:00:00	25.000	40.000
node_3	2022-10-01 11:30:00	25.500	41.000
node_3	2022-10-01 12:00:00	26.000	42.000
node_3	2022-10-01 12:30:00	26.500	43.000
node_4	2022-10-01 11:00:00	27.000	38.000
node_4	2022-10-01 11:30:00	27.500	39.000
node_4	2022-10-01 12:00:00	28.000	40.000
node_4	2022-10-01 12:30:00	28.500	41.000
node_5	2022-10-01 11:00:00	18.000	55.000
node_5	2022-10-01 11:30:00	18.500	56.000
node_5	2022-10-01 12:00:00	19.000	57.000
node_5	2022-10-01 12:30:00	19.500	58.000

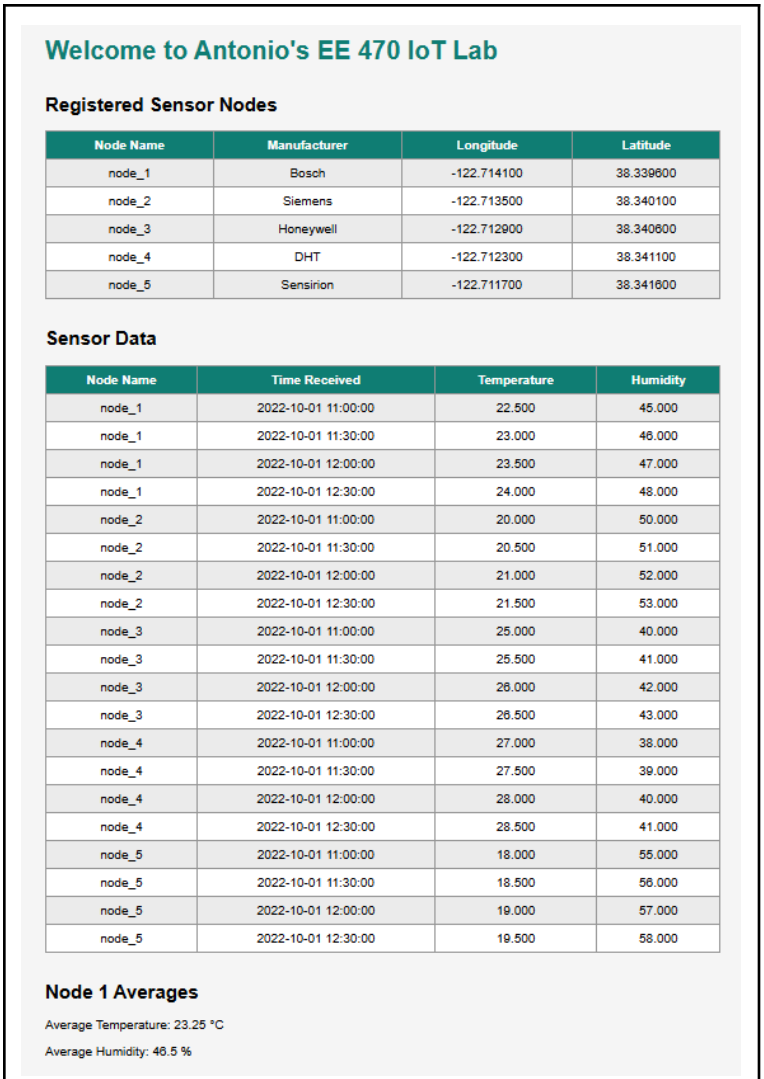
Figure 2: Screenshot of display_data.php

https://antoniokassis.com/display_data.php

Link to your website

2- Insert Data

Modify your PHP file such that every time the page is displayed you also get an average of all the values that are in the database for one of the nodes. Choose any node you like. One will be sufficient. In this case, I am choosing Node 1.



Welcome to Antonio's EE 470 IoT Lab

Registered Sensor Nodes

Node Name	Manufacturer	Longitude	Latitude
node_1	Bosch	-122.714100	38.339600
node_2	Siemens	-122.713500	38.340100
node_3	Honeywell	-122.712900	38.340600
node_4	DHT	-122.712300	38.341100
node_5	Sensirion	-122.711700	38.341600

Sensor Data

Node Name	Time Received	Temperature	Humidity
node_1	2022-10-01 11:00:00	22.500	45.000
node_1	2022-10-01 11:30:00	23.000	46.000
node_1	2022-10-01 12:00:00	23.500	47.000
node_1	2022-10-01 12:30:00	24.000	48.000
node_2	2022-10-01 11:00:00	20.000	50.000
node_2	2022-10-01 11:30:00	20.500	51.000
node_2	2022-10-01 12:00:00	21.000	52.000
node_2	2022-10-01 12:30:00	21.500	53.000
node_3	2022-10-01 11:00:00	25.000	40.000
node_3	2022-10-01 11:30:00	25.500	41.000
node_3	2022-10-01 12:00:00	26.000	42.000
node_3	2022-10-01 12:30:00	26.500	43.000
node_4	2022-10-01 11:00:00	27.000	38.000
node_4	2022-10-01 11:30:00	27.500	39.000
node_4	2022-10-01 12:00:00	28.000	40.000
node_4	2022-10-01 12:30:00	28.500	41.000
node_5	2022-10-01 11:00:00	18.000	55.000
node_5	2022-10-01 11:30:00	18.500	56.000
node_5	2022-10-01 12:00:00	19.000	57.000
node_5	2022-10-01 12:30:00	19.500	58.000

Node 1 Averages

Average Temperature: 23.25 °C

Average Humidity: 46.5 %

Figure 3: Screenshot of the average

Insert data into your database by using the GET method (e.g., CURL command). You can use your browser and use something like this: https://ff.net/firstphp_db_insert.php?nodeId=node_3&timeReceived=12-2-2024 12:00:32&nodeTemp=34 (not a working link - just an example; see the lecture for more information)

Make sure the data is accepted.

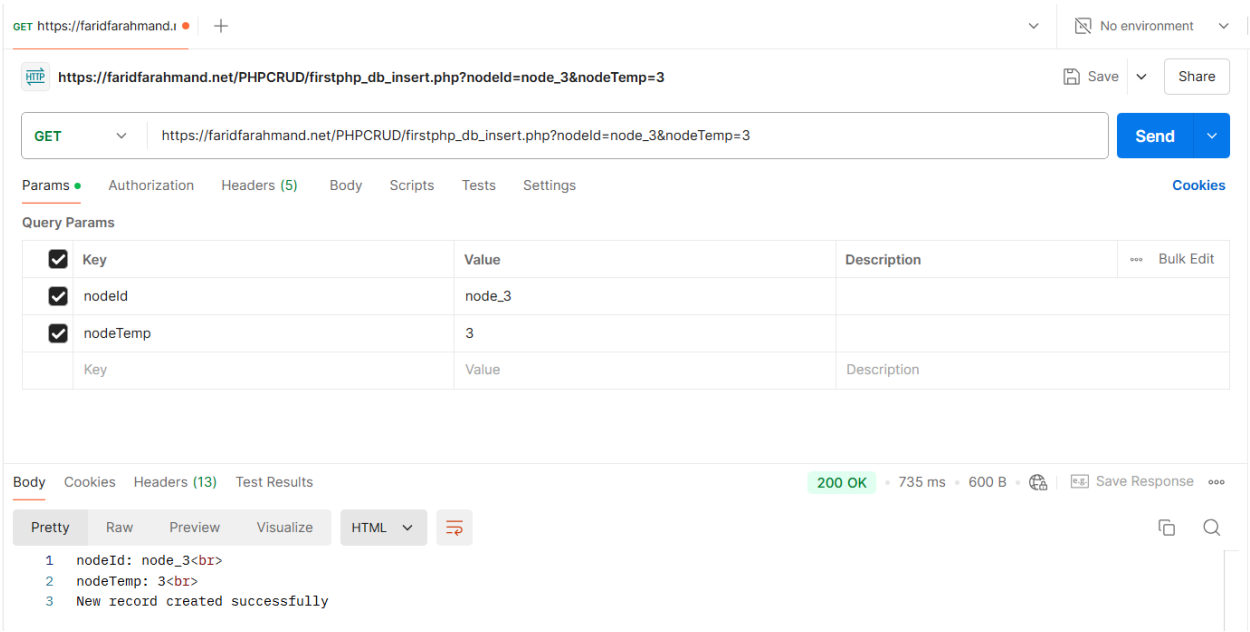
Important requirements:

1. You can have multiple entries for each registered node.
2. If a node is not registered no new data should be received.
3. No duplicate date (**entries with the same exact time**) for the same node must be accepted to the database.
4. When displaying the values, data entries must be sorted according to the node's name.
5. If the values are not within the accepted range the data must not be accepted. See Section 1.2 regarding the data constraints.
6. **If the time is not included in the URL it must automatically be time stamped by the database**.
7. 5- Using the CURL command you should be able to send data to your database.

2.1 Using POSTMAN

POSTMAN is a great tool to execute different methods of HTTP (GET, PUT, POST, etc.). Let's use POSTMAN to inject data into our database.

Download POSTMAN (https://www.postman.com/downloads/). Using GET method send data to your server.



DataBase Assignment

Answer the following questions:

1- Do you see the returned data? Y/N?

Yes

2- Can you use POST to send the data? Y/N?

No

3- Can you use PUT to send the data? Y/N?

No

4- Using POST method select HEADER and then CODE. Can you see the Header Request sent to the Server? Take a snapshot of the header.



Figure 4: Screenshot of the header

5- How long did it take to send your request and get a response to/from the database?

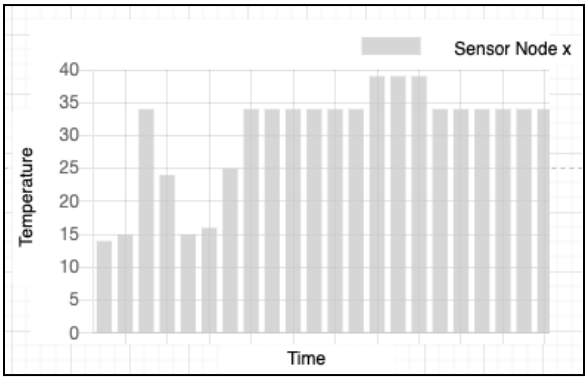
Your Answer: **155.76 ms**

3- Plot Data

In this section we would like to plot the data from the database. Before attempting this part make sure you review the lecture slides. Also, make sure you have downloaded the Chart.js from the course GitHub site (https://github.com/faridfarahmand/SSU/tree/main/IoTCourse/Chartjs).

DO NOT BLINDLY COPY AND PASTE CODES FROM AI SITES! If you cannot answer questions about your project during your demonstration you will not receive a grade. Each incomplete demo will receive -15 points.

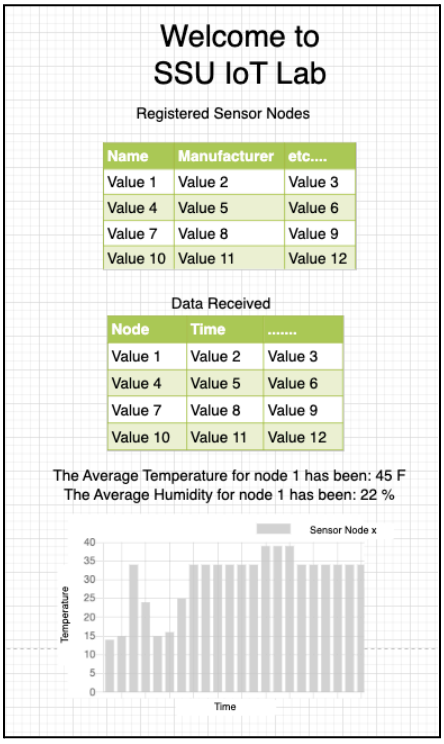
3.1 Plot the data for **only one of your sensors**. The x-axis must be time and the y-axis must represent the temperature. Note that the tile of your graph must be Sensor Node …., as shown below.



Add the new graph to your original webpage:

DataBase Assignment

EE 470 - Introduction to the Internet of Things- Fall / Version: V2



In your page you need to enter the following statistics:
Average Temperature, Average Humidity, Max Temperature, Minimum Humidity.
Make sure your stats are correct. Check them before your demo!

3.2 Add a pull down menu such that the user can select a node and visualise the data for that particular node. When you select the appropriate node all the statistics must be changed accordingly.



Make sure your stats are still showing the correct values.

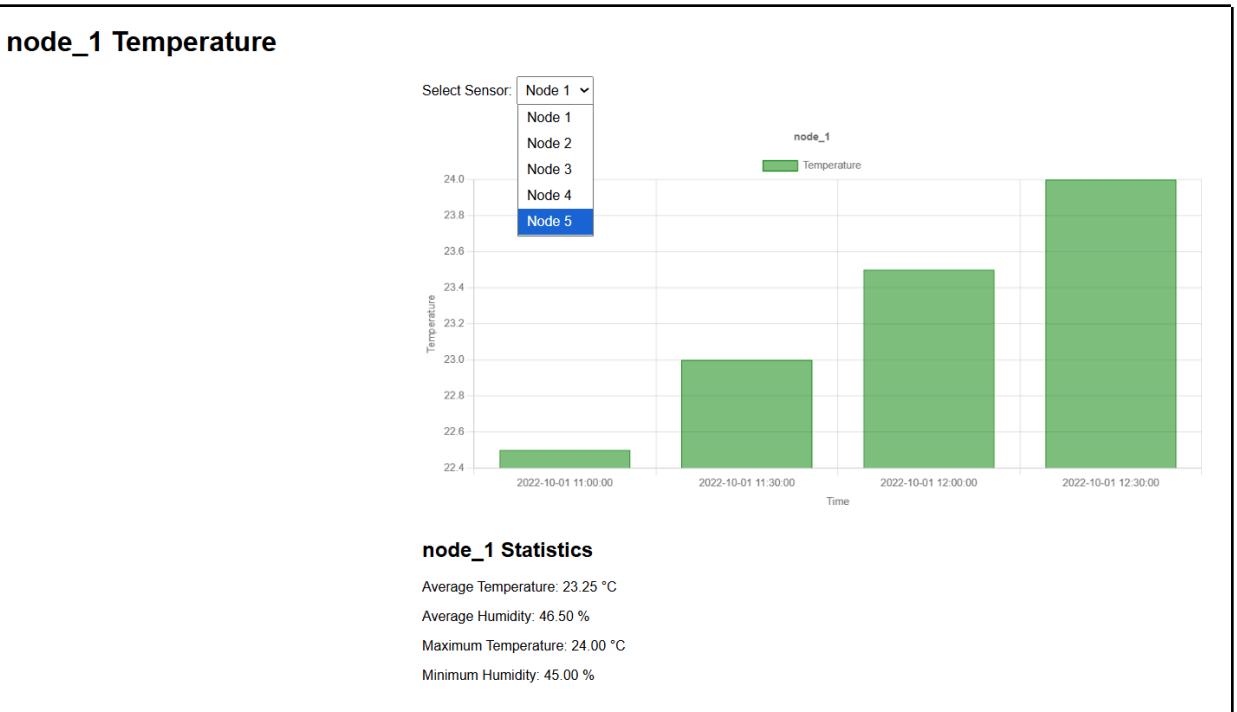


Figure 5: Screenshot of the fully implemented graph

3.3 QUESTIONS:

1- How can you change the bar graph to a line graph? What did you change?

You can change it by replacing 'type: "bar"' with 'type: "line"'. This is in app.js.

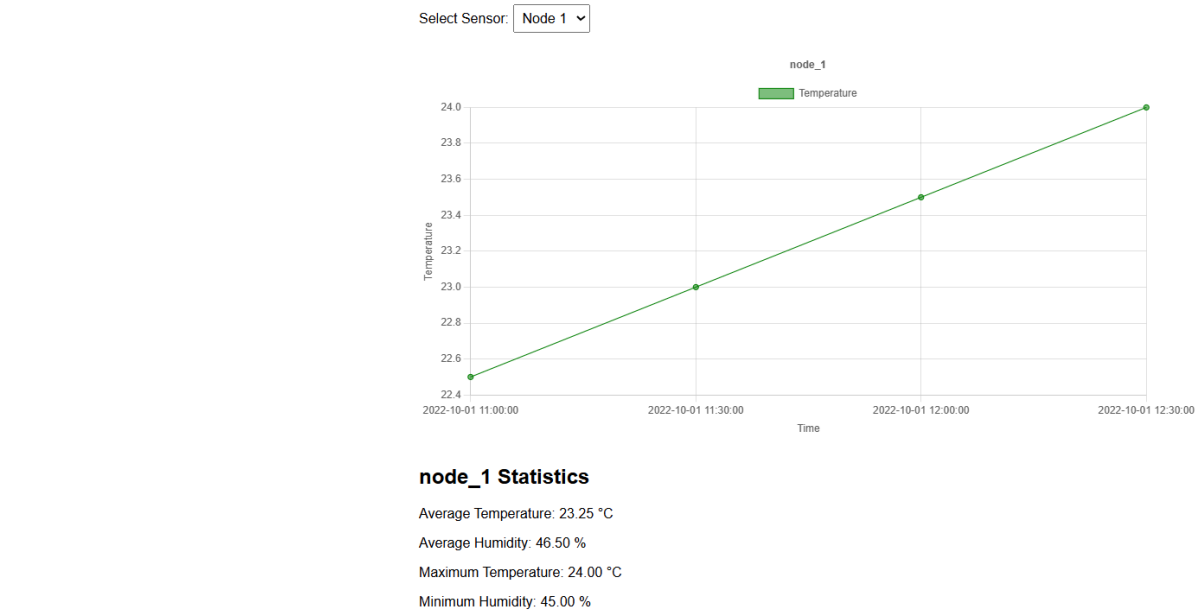


Figure 6: Screenshot of the graph with a line instead of bars

2- Change the graph such that it is GREEN. You can use the rgbs color scheme here: https://rgbacolorpicker.com/. What did you change?

In app.js, you need to change the background color to 'backgroundColor: "rgba(0, 128, 0, 0.5)"' which is green.

3- Show one of your node's data in JSON format.


```
{"node":"node_1","sensorData":[{"node_name":"node_1","time_received":"2022-10-01 11:00:00","temperature":"22.500","humidity":"45.000"},
{"node_name":"node_1","time_received":"2022-10-01 11:30:00","temperature":"23.000","humidity":"46.000"},
{"node_name":"node_1","time_received":"2022-10-01 12:00:00","temperature":"23.500","humidity":"47.000"},
{"node_name":"node_1","time_received":"2022-10-01 12:30:00","temperature":"24.000","humidity":"48.000"}],"stats":
{"avg_temperature":"23.2500000","avg_humidity":"46.5000000","max_temperature":"24.000","min_humidity":"45.000"}}
```


Figure 7: Screenshot of the node data in JSON format

4- Encryption

Change your PHP program in the previous section such that the program receives a BASE64 encoded message and then it should be able to parse it correctly. For example, you should be able to send the following message and your program must parse correctly and enter the appropriate values in the database.

URL Inquery:
https://faridfarahmand.net/PHPCRUD/firstphp_db_insert.php?bm9kZUlkPW5vZGVfMyZub2RlVGVtcD0zNCZ0aW1lUmVjZWl2ZWQ9MjAyMi0xMC0wMSUyMDEwOjI1OjAx

Results:
nodeId=node_3
nodeTemp=34
timeReceived=2022-10-0 2010:25:01

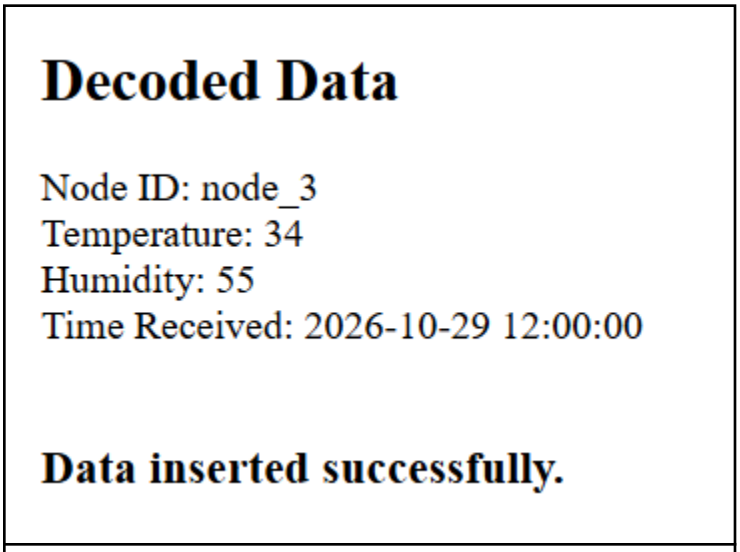


Figure 8: Screenshot of decoded base64 data

Note: You must include Time as part of the URL!

https://antoniokassis.com/insert_base64.php
Link to your page

https://antoniokassis.com/insert_base64.php?bm9kZUlkPW5vZGVfMyZub2RlVGVtcD0zNCZub2RlSHVtaWRpdHk9NTUmdGltZVJlY2VpdmVkPTIwMjYtMTAtMjkgMTI6MDA6MDA=

Sample URL with encoded parameters and the displayed variables on the browser

insert_base64.php

public_html > insert_base64.php

```
17
18
19  // ---------------------------------
20  // Get Base64 encoded query string
21  // ---------------------------------
22
23  $encoded = $_SERVER['QUERY_STRING'];
24
25  if (empty($encoded)) {
26      die("ERROR: No encoded data received.");
27  }
28
29
30  // ---------------------------------
31  // Decode Base64
32  // ---------------------------------
33
34  $decoded = base64_decode($encoded, true);
35
36  if ($decoded === false) {
37      die("ERROR: Invalid Base64 message.");
38  }
39
40
41  // ---------------------------------
42  // Parse decoded values
43  // ---------------------------------
44
45  parse_str($decoded, $values);
46
47  $nodeId = $values["nodeId"] ?? null;
48  $nodeTemp = $values["nodeTemp"] ?? null;
49  $nodeHumidity = $values["nodeHumidity"] ?? null;
50  $timeReceived = $values["timeReceived"] ?? null;
51
52
53  // ---------------------------------
```

Figure 9: Screenshot of the base64 decoding portion of the code

DataBase Assignment

https://github.com/Neo4724/IoT_EE_470/tree/main/EE%20470%20-%20Assignment%204%3A%20Part%202%20-%20[PHP%20%26%20Databasing]
Place all your codes on your GITHUB page and add the link here.

NOTE: All projects must be shown in class! If you cannot demonstrate your project, you do not receive a grade!

5- Demonstration Requirements

Your demonstration will be assessed based on the following.

1. When correct data is inserted the values must be inserted into the database, displayed on the table in a sorted way (sorted by name name and then time) and then plotted properly. https://ff.net/firstphp_db_insert.php?nodeId=node_3&timeReceived=12-2-2024 12:00:32&nodeTemp=34
2. For the purpose of the demonstration, the microcontroller should always send the time.
    a. If there is no time in the URL, the received data must be automatically time stamped: https://ff.net/firstphp_db_insert.php?nodeId=node_3&nodeTemp=34
3. If the URL includes a node that is not registered the database, plot, and table must not be updated.
4. If the URL includes values that are out of range then the database, plot, and table must not be updated.
5. Make sure you can register a new node and then it will accept new data from the URL.
6. Make sure you can send data using CURL command from the terminal
7. **Make sure you can clearly explain how your code works! You will not receive a grade if you cannot clearly explain how your code operates! Review your code before demonstrating.**
8. **REVIEW YOUR CODE CAREFULLY!**

6. Conclusions

In two-three paragraphs in your own words explain what you learned in this activity. What was interesting?

In this assignment, I learned how to create and manage a database using MariaDB and phpMyAdmin. I created tables for sensor registration and sensor data, added primary and foreign keys, inserted values, and created a view that combined information from multiple tables. I also learned how database constraints can prevent invalid data, such as data from an unregistered sensor or values outside of the allowed range.

In Part 2, I learned how to connect a PHP webpage to the database and display the sensor information. I also used GET requests, CURL, and Postman to send new sensor values to the database. Using Chart.js, I created a graph of the temperature data and

added a dropdown menu that allowed different sensor nodes to be selected while updating the graph and statistics.

The most interesting part of the assignment was seeing how all of the different parts worked together. The database stored the information, PHP communicated with the database, and JavaScript and Chart.js displayed the results on the webpage. I also learned how Base64 encoding can be used to send encoded information to a PHP program and then decode the values before adding them to the database.

7. Quick Survey

Answer the following questions, please:

Survey question	Response
On a scale of 1-10 how did you like this exercise? (10 is the best, 1 is the worst)	8
On a scale of 1-10 how much did you learn as a result of completing this exercise? (10 = plenty; 1=very little)	8
How many hours did you spend completing this exercise?	7

Appendix A

Welcome to SSU IoT Lab

Registered Sensor Nodes

node_name	manufacturer	longitude	altitude
node_1	google	50.726024000000	-6.619468000000
node_2	amazon	51.607783000000	59.869286000000

Data Received

node_name	time_received	temperature	humid
node_1	2022-10-03 11:11:20	40	50
node_2	2022-10-02 11:11:11	30	80
node_1	2022-10-04 11:11:11	44	70
node_1	2022-10-05 10:05:33	34	40
node_2	2022-10-17 10:45:33	38	70
node_1	2022-10-22 02:05:03	34	66
node_2	2022-10-22 02:05:03	34	66
node_1	2022-10-23 02:05:03	45	66
node_1	2022-10-22 11:11:11	44	60
node_1	2022-10-23 11:11:23	50	50
node_2	2022-10-23 11:11:23	50	50
node_2	2022-10-23 11:11:23	20	70
node_2	2022-10-23 09:00:00	26	76

The Average Temperature for node_1 has been: 41.57142857142857F
The Average Humidity for node_1 has been: 57.42857142857143%

